EDUDEBUG

Integración de una base de datos NoSQL: identificación, estructura y justificación de los datos

Asignatura: Nuevas Tecnologías

Entrega: Persistencia políglota (SQL + NoSQL)

Integrantes: [completar nombres del equipo]

Docente: [completar]

Fecha: octubre de 2026

Resumen. EDUDEBUG es una plataforma para aprender HTML básico. En la fase anterior toda la información vivía en un modelo relacional de 11 tablas. Este documento explica qué parte de la problemática se beneficia de una base de datos documental (MongoDB), cómo se estructuran esos datos, por qué no encajan en tablas y cómo conviven con el modelo relacional en una arquitectura de persistencia políglota.

1. Contexto y alcance

La aplicación (backend Flask, frontend HTML/Bootstrap) ofrece un diagnóstico adaptativo de cinco preguntas, un depurador de código HTML que explica los errores encontrados, recomendaciones de repaso y un panel de progreso para el profesor. Los datos centrales (cuentas, cursos, módulos, tests, envíos de código, errores detectados y recomendaciones) son estructurados, tienen relaciones claras y requieren integridad referencial; por eso permanecen en la base relacional (SQLite).

Al escalar la aplicación aparecen dos tipos de información que no se ajustan bien a ese modelo rígido: la **actividad que genera cada usuario al usar la plataforma** y los **comentarios que los estudiantes dejan en los módulos**. Son esos dos conjuntos los que se llevan a MongoDB.

2. Identificación de datos no estructurados

Se evaluó cada dato según cuatro criterios: volumen de escritura, variabilidad de sus campos, necesidad de relaciones e integridad, y forma de consulta.

Dato	Volumen	Variabilidad	Relaciones / integridad	Destino
Cuentas, cursos, módulos	Bajo	Ninguna (campos fijos)	Alta: llaves foráneas, UNIQUE	SQL
Tests diagnósticos y preguntas	Medio	Baja	Alta: test → preguntas → opciones	SQL
Envíos de código y errores detectados	Medio	Baja	Alta: se cruzan para progreso y recomendaciones	SQL
Recomendaciones	Bajo	Baja	Alta: referencian módulos y tipos de error	SQL
Bitácora de actividad	**Alto**: un evento por acción	**Alta**: cada tipo de evento guarda campos distintos	Baja: solo referencia al usuario	**NoSQL**
Comentarios de módulos	Medio-alto, creciente	**Alta**: etiquetas, fragmento de código y reacciones opcionales	Baja: referencia usuario y módulo	**NoSQL**

2.1 Por qué la bitácora de actividad no encaja en tablas

Cada acción del usuario genera un evento: iniciar sesión, responder una pregunta del diagnóstico, enviar código al depurador, ver un tema, comentar. Lo que interesa guardar de cada evento es diferente. Un envío al depurador registra cuántos errores hubo y de qué tipos; el cierre de un diagnóstico registra aciertos y nivel; un tema visto registra su número. En un modelo relacional habría que elegir entre una tabla con muchas columnas casi siempre vacías, una tabla por tipo de evento (más tablas cada vez que se agrega un evento) o una columna de texto sin estructura. En un documento, en cambio, el campo `detalle` simplemente contiene lo que corresponde a ese evento.

Además, es el dato de mayor volumen y casi solo se inserta y se lee por usuario y fecha. No participa en joins complejos ni necesita llaves foráneas, de modo que sacarlo de la base relacional la mantiene

liviana.

2.2 Por qué los comentarios no encajan en tablas

Un comentario básico tiene texto, autor y fecha, pero puede tener además etiquetas (de cero a cinco), un fragmento de código y reacciones (*útil*, *gracias*, *confuso*) que aparecen solo cuando alguien las usa. Modelarlo con tablas exigiría una tabla de etiquetas, una de reacciones y otra para el código, con sus uniones. En MongoDB el comentario es un único documento que ya contiene todo y puede cambiar de forma sin modificar el esquema.

3. Motor elegido: MongoDB

Se eligió MongoDB, una base de datos documental, por los motivos siguientes:

• **Esquema flexible:** los documentos de una misma colección pueden tener campos diferentes, que es exactamente el caso del detalle de los eventos y de los atributos opcionales de los comentarios.

• **Escrituras simples y rápidas:** insertar un evento es una sola operación sobre un documento.

• **Operadores que encajan con el problema:** `$inc` para sumar reacciones (el campo se crea la primera vez), `$unset` para quitar campos opcionales y `$group` para contar eventos por tipo.

• **Disponibilidad gratuita:** MongoDB Atlas ofrece un clúster gratuito que se conecta desde la aplicación con una cadena de conexión (variable de entorno `MONGO_URI`).

4. Estructura de los datos NoSQL

Se definen dos colecciones en la base `edudebug`.

4.1 Colección actividad_usuario

Campo	Tipo	Descripción
_id	ObjectId	Identificador generado por MongoDB.
idUsuario	entero	Referencia lógica a usuarios.id_usuario (SQL).
tipo	texto	registro, login, curso_consultado, tema_visto, diagnostico_iniciado, diagnostico_respuesta, diagnostico_finalizado, codigo_analizado, comentario_creado.
fecha	fecha UTC	Momento del evento; se usa para ordenar.
detalle	documento	Campos propios de cada tipo de evento (ver tabla 4.2). Máximo 2000 caracteres serializados.

Tabla 4.2. Contenido del campo detalle según el tipo de evento.

Tipo de evento	Campos de detalle
login, registro	rol
curso_consultado	idCurso
tema_visto	tema
diagnostico_iniciado	idTest
diagnostico_respuesta	idTest, idPregunta
diagnostico_finalizado	idTest, aciertos, total, nivel
codigo_analizado	idEnvio, totalErrores, tipos (lista), recomendacionGenerada
comentario_creado	idModulo, idComentario

Ejemplo de documento:

```
{
  "_id": ObjectId("6adb7de103aab2216760b66a"),
  "idUsuario": 1,
  "tipo": "codigo_analizado",
  "fecha": ISODate("2026-10-23T15:31:45.711Z"),
  "detalle": {
    "idEnvio": 2,
    "totalErrores": 4,
    "tipos": ["atributo_sin_comillas", "etiqueta_en_mayusculas", "etiqueta_sin_cerrar"],
    "recomendacionGenerada": true
  }
}
```

4.3 Colección comentarios_modulo

Campo	Tipo	Obligatorio	Descripción
_id	ObjectId	Sí	Identificador generado por MongoDB.
idUsuario	entero	Sí	Autor; referencia lógica a usuarios.id_usuario (SQL).
idModulo	entero	Sí	Módulo comentado; referencia lógica a modulos.id_modulo (SQL). Debe existir al crear.
texto	texto	Sí	Hasta 1000 caracteres.
etiquetas	lista de textos	Sí (puede ir vacía)	Hasta 5, de máximo 30 caracteres; se guardan en minúsculas.
fragmentoCodigo	texto	No	Hasta 2000 caracteres. Si se edita sin enviarlo, el campo se elimina.
reacciones	documento	Sí (inicia vacío)	Contadores por tipo: util, gracias, confuso. Cada campo aparece al recibir la primera reacción.
fecha	fecha UTC	Sí	Creación.
editadoEn	fecha UTC	No	Solo existe si el comentario fue editado.

Ejemplo de documento:

```
{
  "_id": ObjectId("6adb7de103aab2216760b66c"),
  "idUsuario": 1,
  "idModulo": 2,
  "texto": "¿Por qué el DOCTYPE va primero?",
  "etiquetas": ["duda", "estructura"],
  "fragmentoCodigo": "<!DOCTYPE html>",
  "reacciones": { "util": 2, "gracias": 1 },
  "fecha": ISODate("2026-10-23T15:31:45.715Z")
}
```

5. Relación entre SQL y NoSQL

El vínculo entre ambas bases es **lógico**: los documentos guardan los identificadores numéricos `idUsuario` e `idModulo` de las tablas `usuarios` y `modulos`. MongoDB no verifica esa referencia, por lo que la validación se hace en la capa de servicio: por ejemplo, un comentario solo se crea si el módulo existe en SQLite y el usuario sale del token de sesión.

Para no duplicar información, **el nombre del autor no se almacena en el comentario**. Al listar los comentarios de un módulo, la aplicación lee los documentos en MongoDB y consulta en SQLite el nombre de cada autor. Si la cuenta ya no existe, se muestra «Usuario eliminado».

Consecuencia de diseño: no hay transacciones que abarquen las dos bases. SQLite es la fuente de verdad; la bitácora es secundaria. Por eso el registro de actividad nunca interrumpe la operación principal: si MongoDB no está configurado o falla, el evento se omite y se deja una advertencia en el registro del servidor.

6. Índices y consultas

Colección	Índice	Consulta que acelera
actividad_usuario	(idUsuario ascendente, fecha descendente)	Últimas N actividades de un usuario.
actividad_usuario	(tipo)	Filtrar o contar por tipo de evento.
comentarios_modulo	(idModulo, fecha descendente)	Comentarios de un módulo, del más reciente al más antiguo.
comentarios_modulo	(idUsuario)	Contar los comentarios de un usuario.

Los índices se crean automáticamente la primera vez que la aplicación usa las colecciones. Además se usa una agregación para obtener el total de eventos por tipo de un usuario:

```
db.actividad_usuario.aggregate([
  { $match: { idUsuario: 1 } },
  { $group: { _id: "$tipo", total: { $sum: 1 } } }
])
```

7. Persistencia políglota en la aplicación

La aplicación consulta cada base según el tipo de dato:

Funcionalidad	SQL (SQLite)	NoSQL (MongoDB)
Inicio de sesión y registro	Cuenta y contraseña cifrada	Evento login / registro
Diagnóstico adaptativo	Test, preguntas, opciones y nivel	Eventos de inicio, respuestas y cierre
Depurador de código	Envío, errores detectados, recomendaciones	Evento codigo_analizado con su resumen
Módulos del curso	Contenido y orden	Comentarios con etiquetas y reacciones
Perfil del usuario	Datos, progreso, recomendación	Últimas 10 actividades y totales por tipo

7.1 Lógica de negocio que vincula ambos tipos de datos

El endpoint `GET /api/usuarios/{id}/perfil-completo` entrega en una sola respuesta el perfil estructurado del usuario (SQL) junto con sus últimas 10 actividades, el total de eventos por tipo y la cantidad de comentarios escritos (NoSQL). Puede consultarlo el propio usuario o un profesor. Si MongoDB no está disponible, la parte SQL se entrega igual y el bloque `actividad` indica `disponible: false`.

```
{
  "usuario":  { "id": 1, "nombre": "...", "email": "...", "rol": "estudiante" },
  "progreso": { "porcentajeCurso": 0, "erroresCorregidos": 0, "totalEnvios": 2, ... },
  "recomendacion": { "modulosSugeridos": [ ... ], "basadoEnErrores": [ ... ] },
  "actividad": {
    "disponible": true,
    "ultimas": [ { "tipo": "comentario_creado", "fecha": "...", "detalle": { ... } }, ... ],
    "totalPorTipo": { "login": 1, "codigo_analizado": 2, "comentario_creado": 1 },
    "comentariosEscritos": 1
  }
}
```

Un segundo caso es `GET /api/modulos/{id}/comentarios`: los comentarios provienen de MongoDB y el nombre del autor, de SQLite. En la página, el perfil del estudiante muestra su actividad reciente a partir de este endpoint.

8. Operaciones CRUD sobre MongoDB

Operación	Endpoint	Colección	Regla de acceso
Crear	POST /api/actividad	actividad_usuario	Solo tipo tema_visto; el resto lo registra el servidor.
Leer	GET /api/actividad?limite=&tipo;=	actividad_usuario	Solo la propia actividad (límite de 1 a 100).
Eliminar	DELETE /api/actividad/{id}	actividad_usuario	Solo el dueño del evento.
Crear	POST /api/modulos/{id}/comentarios	comentarios_modulo	Usuario con sesión; el módulo debe existir.
Leer	GET /api/modulos/{id}/comentarios · GET /api/comentarios/{id}	comentarios_modulo	Usuario con sesión.
Actualizar	PUT /api/comentarios/{id}	comentarios_modulo	Solo el autor.
Actualizar (parcial)	POST /api/comentarios/{id}/reacciones	comentarios_modulo	Usuario con sesión; operador $inc.
Eliminar	DELETE /api/comentarios/{id}	comentarios_modulo	El autor o un profesor (moderación).

9. Decisiones y limitaciones

• **Validación en la aplicación:** MongoDB no impone un esquema en este diseño; los tipos, longitudes y valores permitidos se validan en los controladores antes de insertar.

• **Consistencia entre bases:** al no existir transacciones entre SQLite y MongoDB, la bitácora puede quedar sin un evento si Mongo falla en ese momento. Se aceptó porque la bitácora es informativa.

• **Datos personales:** en MongoDB solo se guardan identificadores numéricos, texto de comentarios y metadatos del evento. No se almacenan contraseñas ni correos.

• **Crecimiento de la bitácora:** la implementación actual no elimina eventos antiguos. Como mejora futura (no implementada) se propone un índice con vencimiento automático (TTL) sobre el campo fecha.

• **Integridad referencial:** si en el futuro se permite eliminar cuentas o módulos en SQL, habrá que limpiar también sus documentos en MongoDB, porque la base documental no lo hace por sí sola.

10. Conclusiones

La separación por tipo de dato aprovecha lo mejor de cada motor: el modelo relacional conserva los datos estructurados que requieren integridad y relaciones, y MongoDB absorbe los eventos de alta frecuencia y los comentarios de forma variable sin obligar a modificar tablas. El endpoint de perfil completo y el listado de comentarios demuestran que ambas bases pueden consultarse de manera coordinada dentro de una misma funcionalidad, con una degradación controlada cuando la base documental no está disponible.